



JavaScrip
Logo

JAVASCRIPT SERIES

Day 10 Mastery Course

Scope, Hoisting & Closures

Core Execution Mechanics & Memory Preservation in JS



Instructor: Mahira Noor



0:00 – 1:00

Session Overview

Three Interconnected Pillars of JavaScript

0:00 – 1:00 | Slide 02



1. Scope

Defines variable visibility and boundaries across execution contexts.



2. Hoisting

Creation phase behavior where declarations are registered first.



3. Closures

Functions retaining lexical scope state after outer execution finishes.

💡 These three topics directly power 90% of JavaScript conceptual technical interview questions.

What is Scope?

1:00 – 3:00 | Slide 03

Scope = variable kahan access ho sakta hai

Core Definition

Scope defines the accessibility (visibility) of variables within different sections of code execution.

- 🛡 Every variable has strict boundary limits.
- 🚫 Code outside the boundary cannot see or access inside variables.

Real-World Analogy



Inside House (Home Scope)

✅ Wi-Fi Password Connected ✅



Outside Street

❌ Out of Signal Range ❌

3 Types of Scope

Global vs Function vs Block Scope

3:00 – 5:00 | Slide 04

1. Global Scope

File Wide

Declared outside functions/blocks;
accessible everywhere in the file.

2. Function Scope

Inside Function

Declared inside a function; accessible
strictly inside that function.

3. Block Scope

Inside { }

Declared inside `{ }` curly braces (if, loops);
follows `let` & `const`.



Crucial Industry Note:

`var` does **NOT** follow block scope! Only `let` and `const` respect block boundaries. This is why modern JS avoids `var`.

Today's Roadmap

4 Key Stages of Today's Session

5:00 – 7:00 | Slide 05

- 1 **Global, Function, and Block scope** — Live code demonstrations
- 2 **var vs let** — Deep dive into variable leaking & scope differences
- 3 **Hoisting** — How JS lifts variable and function declarations up in memory
- 4 **Closures** — Functions retaining outer lexical memory

Global Scope

Universal File-Wide Accessibility

7:00 – 11:00 | Slide 06

Concept Breakdown

Variables declared outside any function or code block exist in the **Global Scope**. They are loaded into memory when the script starts and remain accessible throughout the script's lifetime.



Universal Access

Accessible from inside any function or nested block.



Memory Persistence

Stays active in memory as long as the page/app is running.

Global Scope — Code Example

Live Code Execution

7:00 – 11:00 | Slide 07

```
let siteName = "MyWebsite"; // Global scope function showName() { console.log(siteName); }
```

MyWebsite
CONSOLE OUTPUT: MyWebsite
MyWebsite

Function Scope

Encapsulation Within Function Boundaries

11:00 – 15:00 | Slide 08

Concept Breakdown

Variables declared inside a function belong strictly to that function's scope. They are created when the function executes and destroyed when the function finishes.



Private to Function

Cannot be read or modified from outer global space.



Automatic Garbage Collection

Memory freed automatically after function call completes.

Function Scope — Code Example

Accessing Out-of-Scope Variable

11:00 – 15:00 | Slide 09

```
function greetUser() { let message = "Hello!"; // function scope console.log(message); } g
```

✖ UNCAUGHT REFERENCEERROR: message is not defined

Block Scope — var vs let

Curly Brace Boundaries { }

15:00 – 19:00 | Slide 10

What is a Block?

Any code wrapped inside curly braces { } (e.g., **if** statements, **for** loops) creates a block.

var (No Block Scope)

Ignores block boundaries and leaks into outer scope.

Causes Bugs ❌

let / const (Block Scoped)

Strictly trapped inside { } curly braces.

Safe & Modern ✅

Block Scope — Live Example

Testing Access Outside if Block

15:00 – 19:00 | Slide 11

```
if (true) { let blockVar = "I am inside a block"; var oldVar = "I am using var"; } console
```

```
"I am using var"
```

```
Uncaught ReferenceError: blockVar is not defined
```

Block Scope Comparison & Scope Hierarchy

15:00 – 19:00 | Slide 12

var Leaking vs let Protection

var Behavior

Leaks outside the block ✓ Accessible outside

let Behavior

Stays inside block ✗ ReferenceError outside

SCOPE HIERARCHY DIAGRAM

GLOBAL SCOPE

FUNCTION SCOPE

BLOCK SCOPE {let / const}

Real-World Bug: var vs let in Loops

15:00 – 19:00 | Slide 13

Classic Interview Problem — setTimeout in Loop

```
// Loop 1 with var for (var i = 0; i < 3; i++) { setTimeout(() ⇒ console.log("var i:", i)
```

Loop Bug Comparison

15:00 – 19:00 | Slide 14

Why var Fails and let Succeeds

var Loop Bug ❌

```
var i: 3, var i: 3, var i: 3
```

Reason: `var` shares a single variable binding across all loop iterations. By the time `setTimeout` fires, `i` has reached 3.

let Loop Solution ✅

```
let j: 0, let j: 1, let j: 2
```

Reason: `let` creates a new fresh block scoped variable instance for every single loop iteration.

Hoisting

Variable & Function Memory Allocation

19:00 – 23:00 | Slide 15

Core Definition

Hoisting is JavaScript's default behavior of allocating memory for variable and function declarations before code execution runs.



Creation Phase

JS engine scans declarations and "lifts" them to top of scope memory.



Execution Phase

Values are assigned line by line during actual runtime execution.

var Hoisting Example

Declaration Hoisted, Value Not Assigned

19:00 – 23:00 | Slide 16

```
console.log(myVar); // undefined (not an error!) var myVar = "Hello";
```

CONSOLE OUTPUT: **undefined**

 JavaScript implicitly lifts **var myVar;** to top with **undefined** initialization.

let / const Hoisting & Temporal Dead Zone (TDZ)

19:00 – 23:00 | Slide 17

Why let throws ReferenceError

```
console.log(myLet); // ❌ ERROR! let myLet = "Hello";
```

ReferenceError: Cannot access 'myLet' before initialization

TEMPORAL DEAD ZONE (TDZ) TIMELINE

TOP OF SCOPE

Start of Scope (Hoisted but uninitialized)

● TEMPORAL DEAD ZONE (TDZ)

Accessing variable here throws Error!

DECLARATION LINE

let myLet = "Hello"; (TDZ Ends)

Function Declaration Hoisting

19:00 – 23:00 | Slide 18

Fully Hoisted Functions

```
sayHello(); // Works! "Hello!" function sayHello() { console.log("Hello!"); }
```

OUTPUT: Hello! 

Standard **Function Declarations** are fully hoisted along with their complete function body definition into memory.

Function Expression Hoisting

Arrow Functions & Variables Storing Functions

19:00 – 23:00 | Slide 19

```
sayHi(); // ✗ ERROR! const sayHi = function () { console.log("Hi!"); };
```

✗ **ReferenceError: Cannot access 'sayHi' before initialization**

Function Expressions follow variable hoisting rules. Variable **const** **sayHi** is in TDZ!

Function Hoisting Comparison

19:00 – 23:00 | Slide 20

🔥 Common Interview Question

Function Declaration

Fully Hoisted ✓

Can be invoked anywhere in scope before declaration line.

```
function foo() {}
```

Function Expression

Not Hoisted ✗

Behaves like variable; subject to TDZ error if called early.

```
const foo = () => {}
```

Closures — Introduction

Memory Preservation Across Execution Contexts

23:00 – 30:00 | Slide 21

SIMPLE DEFINITION

A closure gives an inner function access to an outer function's scope variables, even after the outer function has finished executing.



Remembers Memory

Inner function remembers outer scope state.



Data Privacy

Keeps state variables private from global tampering.

Closure Counter Example

Live Code Demonstration

23:00 – 30:00 | Slide 22

```
function outerFunction() { let count = 0; // outer variable function innerFunction() { count++; con
```

Closure Execution Step-by-Step

Progressive Breakdown — Step 1

23:00 – 30:00 | Slide 23

Step 1 `outerFunction()` call hua

Usne `count = 0` banaya, aur `innerFunction` ko return kar diya.

Closure Execution Step-by-Step

23:00 – 30:00 | Slide 24

Progressive Breakdown — Steps 1 & 2

Step 1 `outerFunction()` call hua

Usne `count = 0` banaya, aur `innerFunction` ko return kar diya.

Step 2 `outerFunction` ka kaam khatam ho gaya

`outerFunction` execution stack se nikal gaya, lekin `innerFunction` ke paas ab bhi `count` ki memory baqi hai.

Closure Execution Step-by-Step

Progressive Breakdown — All Steps Complete

23:00 – 30:00 | Slide 25

Step 1 outerFunction() call hua and returned innerFunction.

Step 2 outerFunction ended, but innerFunction retained count memory.

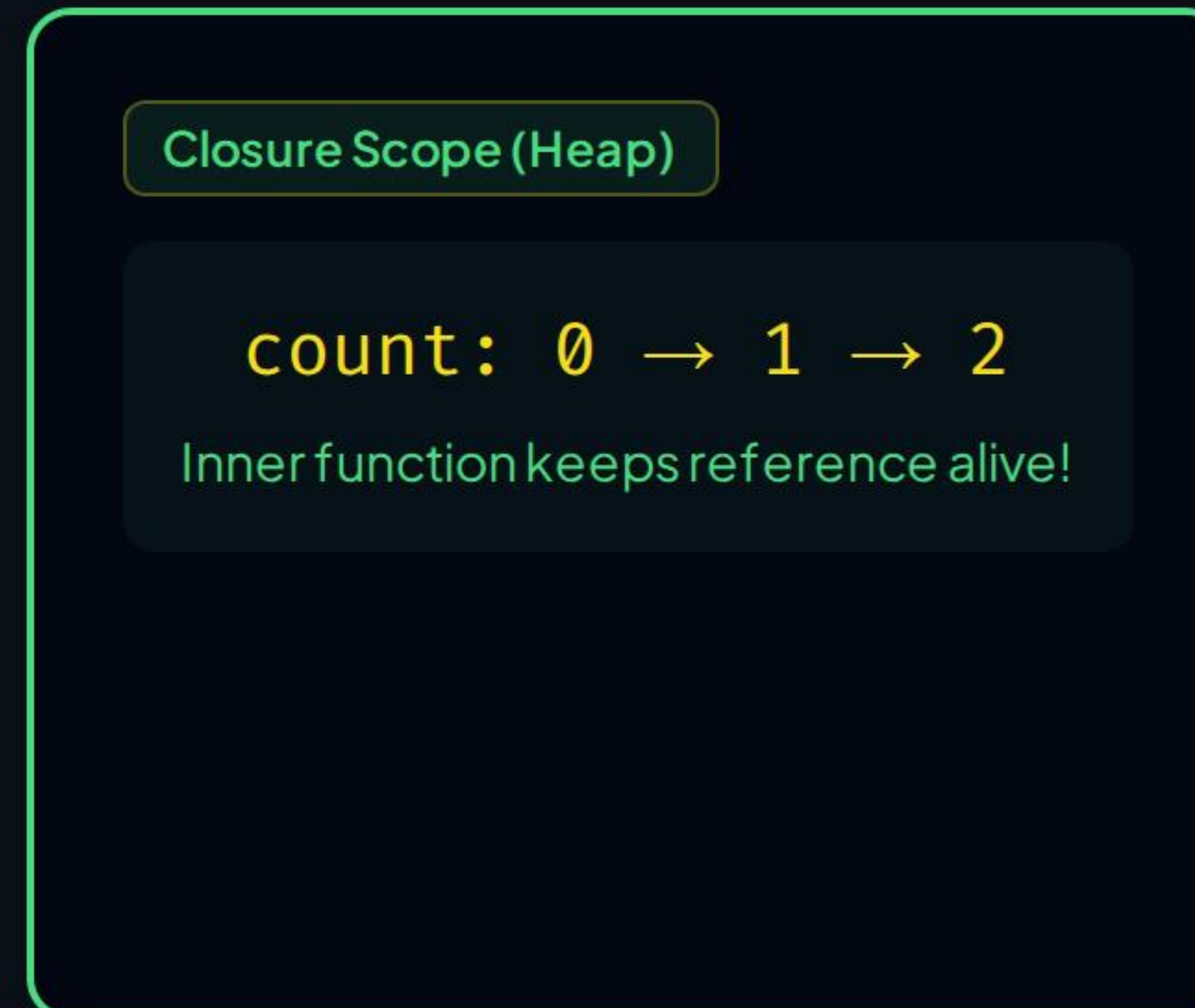
Step 3 **counter() invoke hota hai**

count apni purani value yaad rakhta hai aur har call pe +1 hota rehta hai (1 → 2 → 3).

Closure Memory Preservation Diagram

How JS Keeps Scope Alive in Heap Memory

23:00 – 30:00 | Slide 26



Why Closures Matter — Real World Example

23:00 – 30:00 | Slide 27

Bank Account Encapsulation (Part 1)

```
function createBankAccount(initialBalance) { let balance = initialBalance; // Private Variable return { depos
```

Bank Account Example — Usage

23:00 – 30:00 | Slide 28

Executing Authorized Methods (Part 2)

```
const myAccount = createBankAccount(5000); myAccount.deposit(2000); // Deposited 2000. New
```

```
Deposited 2000. New balance: 7000  
CONSOLE OUTPUT: Withdrew 1000. New balance: 6000  
Current balance: 6000
```

Data Privacy & Encapsulation Visual

Direct Access Prevention

23:00 – 30:00 | Slide 29

```
console.log(myAccount.balance);
```

`undefined` – Private variable!

balance

PROTECTED PRIVATE STATE

deposit()

withdraw()

checkBalance()

Data Privacy & Encapsulation

Key Architectural Benefits

23:00 – 30:00 | Slide 30



Variables cannot be directly modified or corrupted from external scripts.



Controlled access through explicit public methods (`deposit`, `withdraw`).



Mimics real-world banking security and OOP private properties in JS.

Closures in Modern Frontend

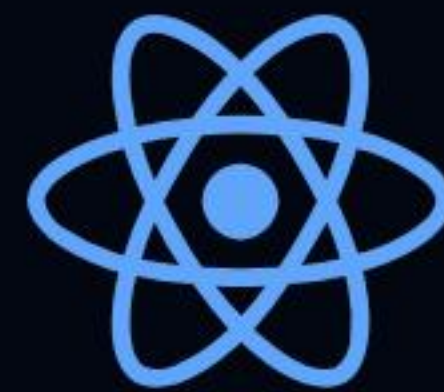
23:00 – 30:00 | Slide 31

Connection to React Hooks

React useState Connection

React's `useState` hook uses JavaScript closures behind the scenes to preserve component state across re-renders!

🎓 Master closures today, and React state will feel effortless tomorrow.



`useState() = Closure`

Quick Comparison Recap Table

Core Concepts Summary Matrix

Concept	Key Point
Global Scope	Poori file mein accessible
Function Scope	Sirf us function ke andar accessible
Block Scope	Sirf {} ke andar accessible (let/const only)
var	Block scope follow nahi karta — bugs create karta hai
Hoisting (var)	Declaration upar utha leta hai, value undefined hoti hai
Hoisting (let/const)	Hoist hote hain, lekin Temporal Dead Zone mein — error dete hain
Closure	Inner function outer function ki variables yaad rakhta hai

Practice Task / Homework

30:00 | Slide 33

Exercise: Build createCounter Function

```
// Build a function called "createCounter" that: // - Has a private variable "count" starting at 0 // - Return
```

Practice Task Requirements

30:00 | Slide 34

 Try It Yourself Challenge

- 1 `count` variable bahar se directly access nahi honi chahiye — sirf closures ke through.
- 2 `createCounter()` ko do dafa call karke 2 independent counters banao (`counter1`, `counter2`).
- 3 Har method call ke baad current count console pe print karo.

 **Share your solution in the comments section!**

Final Session Recap

30:00 | Slide 35

4 Key Takeaways

✓ **Scope** — decides where variables are accessible (global, function, block)

✓ **var vs let** — var ignores block scope and creates subtle loop bugs

✓ **Hoisting** — JS lifts declarations to top of memory before execution

✓ **Closures** — inner functions retain outer variable memory for data privacy

Next Session Preview

Day 11 — DOM Intro

30:00 | Slide 36

Coming Up Next

Day 11 — DOM Intro: Selecting Elements

`</>` `querySelector` & `querySelectorAll`

⚡ `getElementById` & DOM Manipulation

🌐 Interacting with real browser web pages!



Browser DOM



Thank You for Watching!

Like • Comment • Subscribe



Like



Comment



Subscribe

Allah Hafiz, and Keep Coding! 🚀

Image Sources



<https://toppng.com/uploads/preview/javascript-logo-number-angularjs-node-javascript-logo-11563241338go76tq0nxz.png>

Source: toppng.com